

COMPTE RENDU- BATAILLE NAVALE



Sommaire

Introduction

PARTIE 1 : ARCHITECTURE ET INITIALISATION

Étape 1 : Initialisation des structures de données

PARTIE 2 : PHASE DE PRÉPARATION

Étape 2 : Placement des pions du joueur

Étape 3 : Placement aléatoire des pions de l'ordinateur

PARTIE 3 : SYSTÈME D'AFFICHAGE

Étape 4 : Procédure affichagetab()

Étape 5 : Procédure affichagetabCache()

PARTIE 4 : MOTEUR DE JEU

Étape 6 : Système de tir du joueur

Étape 7 : Intelligence artificielle de l'ordinateur

PARTIE 5 : GESTION DE PARTIE

Étape 8 : Boucle principale et conditions de victoire

CONCLUSION

Introduction

Dans ce Compte rendu, on a recréé le Jeu Bataille Navale avec le langage Java. L'objectif était de concevoir et implémenter une version simplifiée du jeu classique de bataille navale en utilisant le langage Java, en se basant sur plusieurs procédures.

PARTIE 1 : ARCHITECTURE ET INITIALISATION

Étape 1 : Initialisation des structures de données

Nous avons créé et initialisé les structures de données fondamentales du jeu. Pour cela, nous avons initialisé un tableau à deux entrées pour les lignes et les colonnes ce qui permettait au joueurs d'entrée les nombres de lignes et de colonnes

PARTIE 2 : PHASE DE PRÉPARATION

Étape 2 : Placement des pions du joueur

Nous allons créer une boucle jusqu'à ce que les 5 bateaux soit placés, établir les limites du tableau, mais aussi empêcher que le joueur place son bateau sur une case déjà occupée.

```

System.out.println("PLACEMENT DE VOS BATEAUX");
do {
    System.out.println("Sur quelle ligne voulez vous placer votre pion ?");
    int batUserL = sc.nextInt();
    System.out.println("Sur quelle colonne voulez vous placer votre pion ?");
    int batUserC = sc.nextInt();

    if (batUserL >= 1 && batUserL <= lignes && batUserC >= 1 && batUserC <= colonnes) {
        int ligneIndex = batUserL - 1;
        int colonneIndex = batUserC - 1;

        // Vérifier si case déjà prise
        if (tabJoueur[ligneIndex][colonneIndex] == 1) {
            System.out.println("Cette case est déjà occupée par un bateau !");
        } else {
            // Placement du bateau
            tabJoueur[ligneIndex][colonneIndex] = 1;
            bateauxPlaces++;
            System.out.println("Le bateau " + bateauxPlaces + " est bien placé en (" + batUserL + "," + batUserC + ") !");

            // Affichage du plateau
            System.out.println("\nVotre plateau de jeu :");
            afficheTabJoueur(tabJoueur, lignes, colonnes);
        }
    } else {
        System.out.println("Vous êtes hors des limites! Veuillez saisir des valeurs entre 1 et " + lignes + " pour les ligne
    }
} while (bateauxPlaces < nombreBateaux);

```

```

Sur quelle ligne voulez vous placer votre pion ?
2
Sur quelle colonne voulez vous placer votre pion ?
2
Le bateau 2 est bien placé en (2,2) !

```

```

Sur quelle ligne voulez vous placer votre pion ?
5
Sur quelle colonne voulez vous placer votre pion ?
5
Cette case est déjà occupée par un bateau !
Sur quelle ligne voulez vous placer votre pion ?

```

Étape 3 : Placement aléatoire des pions de l'ordinateur

Nous avons généré aléatoirement les places pour les bateaux de l'ordinateur tout en s'assurant que l'emplacement choisi est vide.

```

System.out.println("\nL'ordinateur place ses bateaux...");
do {
    int batOrdil = (int)(Math.random() * lignes) + 1;
    int batOrdic = (int)(Math.random() * colonnes) + 1;

    int ligneIndOrdil = batOrdil - 1;
    int colonneIndOrdil = batOrdic - 1;

    // Vérifier si case libre
    if (tabOrdil[ligneIndOrdil][colonneIndOrdil] == 0) {
        tabOrdil[ligneIndOrdil][colonneIndOrdil] = 1;
        bateauxPlacesOrdil++;
    }
} while (bateauxPlacesOrdil < nbBatOrdil);

System.out.println("L'ordinateur a placé tous ses " + nbBatOrdil + " bateaux !");

```

PARTIE 3 : SYSTÈME D'AFFICHAGE

Étape 4 : Procédure affichagetab()

On va afficher un tableau avec tous les bateaux visibles dedans, les cases contenant la le signe "o" seront prises par un bateau et les autres afficheront le signe "~".

```
// ÉTAPE 4 : Procédure d'affichageTab
// Procédure
static void affichagetabjoueur(int[][] tab, int nblignes, int nbColonnes) {
    // Afficher les numéros de colonnes
    System.out.print(" ");
    for (int j = 1; j <= nbColonnes; j++) {
        System.out.print(j + " ");
    }
    System.out.println();

    // Afficher le contenu du tableau
    for (int i = 0; i < nblignes; i++) {
        // Afficher le numéro de ligne
        System.out.print((i + 1) + " ");
        if (i < 9) System.out.print(" ");

        for (int j = 0; j < nbColonnes; j++) {
            if (tab[i][j] == 1) {
                System.out.print("o ");
            } else {
                System.out.print("~ ");
            }
        }
        System.out.println();
    }
}
```

```
Votre plateau de jeu :
 1 2 3 4 5
1  o ~ ~ ~ ~
2  ~ o ~ ~ ~
3  ~ ~ o ~ ~
4  ~ ~ ~ o ~
5  ~ ~ ~ ~ o
```

Étape 5 : Procédure affichagetabCache()

Nous allons afficher un tableau avec "?" pour les cases non découvertes, "o" pour les bateaux touchés et "x" pour les coups dans l'eau

```
Plateau de l'ordinateur :
 1 2 3 4 5
1  x ? ? ? ?
2  ? ? ? ? ?
3  ? ? ? ? ?
4  ? ? ? ? ?
5  ? ? ? ? ?
```

```
for (int i = 0; i < nblignes; i++) {
    // Afficher le numéro de ligne
    System.out.print((i + 1) + " ");
    if (i < 9) System.out.print(" ");

    for (int j = 0; j < nbColonnes; j++) {
        if (tab[i][j] == 0) {
            // aucune action sur cette case → ?
            System.out.print("? ");
        } else if (tab[i][j] == 1) {
            // un pion non découvert → ?
            System.out.print("? ");
        } else if (tab[i][j] == 2) {
            // un pion découvert → o
            System.out.print("o ");
        } else if (tab[i][j] == 3) {
            // case découverte sans pion → x
            System.out.print("x ");
        } else {
            // par défaut
            System.out.print("? ");
        }
    }
    System.out.println();
}
```

PARTIE 4 : MOTEUR DE JEU

Étape 6 : Système de tir du joueur

On demande au joueur de saisir la ligne et la colonne qu'il veut découvrir, on laisse une pause de 2 secondes pour le suspense. Les trois issues possibles sont raté, touché ou tir à blanc.

Choses à faire:

- Saisie des coordonnées avec validation des bornes
- Vérification d'état de la case cible
- Mise à jour de la grille et des compteurs

Étape 7 : Intelligence artificielle de l'ordinateur

L'ordinateur doit générer aléatoirement des coordonnées non encore jouées, nous ajoutons une pause de 2 secondes pour le suspense et affichons le résultat du jeu de l'ordinateur.

Stratégie :

- Aléatoire uniforme sans mémoire
- Évitement des cases déjà explorées
- Complexité moyenne : $O(1)$ après les premiers tirs

```
do {
    caseValide = true;
    ligneOrdi = (int)(Math.random() * lignes) + 1;
    colonneOrdi = (int)(Math.random() * colonnes) + 1;

    int ligneIndexOrdi = ligneOrdi - 1;
    int colonneIndexOrdi = colonneOrdi - 1;

    // Vérifier si case déjà découverte
    if (tabJoueur[ligneIndexOrdi][colonneIndexOrdi] == 2 ||
        tabJoueur[ligneIndexOrdi][colonneIndexOrdi] == 3) {
        caseValide = false;
    }
} while (!caseValide);

System.out.println("L'ordinateur tire en (" + ligneOrdi + "," + colonneOrdi + ")");

// 7.4 Simulation du tir
System.out.println("Tir en cours...");
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    e.printStackTrace();
}

int ligneIndexOrdi = ligneOrdi - 1;
int colonneIndexOrdi = colonneOrdi - 1;

// 7.5 Résultats du tir
if (tabJoueur[ligneIndexOrdi][colonneIndexOrdi] == 0) {
    System.out.println("L'ordinateur a raté !");
    tabJoueur[ligneIndexOrdi][colonneIndexOrdi] = 3;
} else if (tabJoueur[ligneIndexOrdi][colonneIndexOrdi] == 1) {
    System.out.println("L'ordinateur vous a touché !");
    tabJoueur[ligneIndexOrdi][colonneIndexOrdi] = 2;
}
```

```

  . . . . .
TOUR DE L'ORDINATEUR
L'ordinateur tire en (4,2)
Tir en cours...
L'ordinateur a raté !
```

PARTIE 5 : GESTION DE PARTIE

Étape 8 : Boucle principale et conditions de victoire

Dans cette étape on va vérifier si le joueur ou l'ordinateur a trouvé tous les bateaux de son adversaire, puis affiche le résultat final et propose une autre partie.

```
// ÉTAPE 8.1 : Vérifier fin de partie
if (nbPionTrouveJoueur >= nbBatOrdi || nbPionTrouveOrdi >= nombreBateaux) {
    partieEnCours = false;
}

// ÉTAPE 8.2 & 8.3 : Fin de partie et résultat
System.out.println("FIN DE LA PARTIE");
if (nbPionTrouveJoueur > nbPionTrouveOrdi) {
    System.out.println("VOUS AVEZ GAGNÉ !");
} else if (nbPionTrouveOrdi > nbPionTrouveJoueur) {
    System.out.println("L'ORDINATEUR A GAGNÉ !");
} else {
    System.out.println("MATCH NUL !");
}

System.out.println("\nVoulez-vous rejouer ? (o/n)");
rejouer = sc.next();

while (rejouer.equalsIgnoreCase("o") || rejouer.equalsIgnoreCase("oui"));
System.out.println("Merci d'avoir joué !");
sc.close();
```

```
Votre plateau :
  1 2 3 4 5
1  o ? o ? x
2  ? o ? x x
3  x ? ? x ?
4  ? x ? o ?
5  x ? x ? o
L'ordinateur a gagné !
FIN DE LA PARTIE
L'ORDINATEUR A GAGNÉ !

Voulez-vous rejouer ? (o/n)
```

CONCLUSION

Ce compte rendu nous a permis d'approfondir les bases en Java et d'apprendre en détail l'utilisation des fonctions et des procédures.